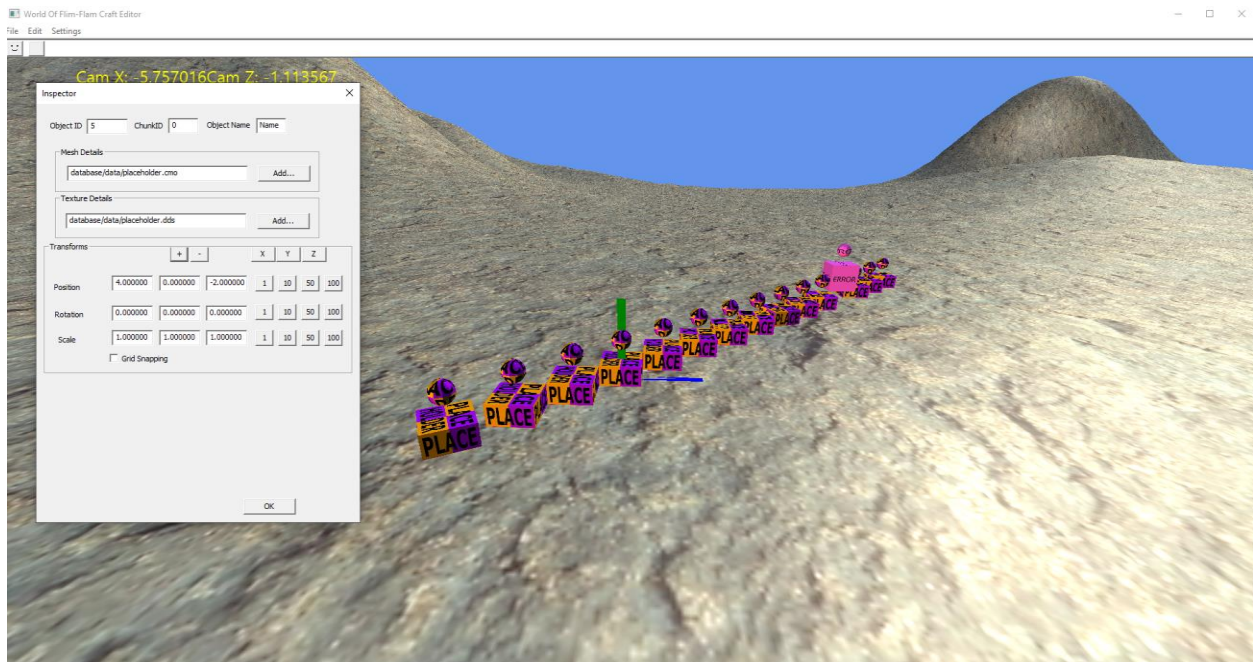


CMP405 Tools Report



Kyle Singer – 2100888

VIDEO LINK: <https://youtu.be/WVnzodSoGLM>

Feature Summary:

Several features have been added to the WOFC Editor for this submission. The features are as follows:

- A camera class has been created which contains all functionality for world movement, including rotation using the mouse while right mouse button is down.
- Mouse picking has been implemented to allow users to click on objects in the scene and add their ID to a vector of ints, this allows for multi-selection of objects.
- The newest object to be selected has a gizmo attached to it which you can use to change its location, rotation and scale by pressing either 1,2 or 3 and dragging from the different gizmo lines.
- An “inspector” dialogue box, which shows the transforms, mesh and texture of an object alongside chunk ID, ID and object name has been created in MFC. There are buttons you can press in the dialogue box to adjust any transform by 1, 10, 50 or 100 respectively.
- Engine settings dialogue box was implemented to allow for changes to be made to the camera controls such as speed, rotation and inverting of controls. These values are then saved into a txt file and reloaded each time the application is started again.
- Undo / Redo was added in through MFC and in editor through use of command interfaces. This stores changes made by the user – transforms as an example and allows the user to revert them with control + z and redo with control + y.
- Delete, cut, copy and paste were also added in as commands.
- Adjusted MFC settings to pass in the new window size and screen centre point to the game class whenever the user resizes the window.

Controls:

- Control + Z = undo
- Control + Y = redo
- Control + X = cut
- Control + C = copy
- Control + V = paste
- Left click = mouse picking + gizmo
- Right Click = enable camera rotation from mouse
- W A S D = Camera movement
- Q = camera up
- E = camera down

MFC features are split into the 'edit' and 'settings' categories in the menu tabs.

- Object Selection: A dialogue box that contains every object in the scene, this allows for the user to select any object rendered.
- Object Inspector Window: This window shows basic information about the currently selected – or most recently selected object. This window allows for adjustment of the object transforms, as well as mesh or texture changes. The window will update the object in real time by building the display list whenever a change is detected.
- Engine Settings Window: This window currently just contains camera variables such as speed, rotation rate and inverting of controls. Once a value has been changed it is saved to a CameraSettings.txt file which is read each time the project loads to set camera values to these saved values.

Editor Settings include:

- Camera: A movable camera using WASD, rotated using the right mouse button and moving the mouse. The E and Q keys raise and lower the camera.
- Mouse Picking: Objects can be selected in the scene by left clicking on them. This has been expanded to include multi-select.
- Gizmo: A gizmo is placed on the most recently selected object and allows you to click on an axis and drag to adjust scale, rotation or location of the object.
- Undo/Redo: A command interface is used to store changes made in the scene to objects and their location, rotation or scale. These changes can then be iterated through with control y and control z to undo or redo different states.
- Cut/Copy/Paste/Delete: the command interface was expanded to allow for additional functionality for cut, copy, paste and deletion of objects, with the added functionality of being able to undo / redo each of these changes.

Camera Class

A camera class has been created to store all camera related functionality, including speed, rotation rate, inversion of controls, movement and rotation. This class has been created to allow for all camera related variables to be encapsulated into the same class, allowing for further expansion or multiple different camera's in future changes. This class currently functions as a way to navigate the editor, without an additional functionality. The camera movement is standard; it makes use of input commands header file which is updated by changes received in toolmain in regard to keypresses and sets booleans to true

or false depending on if a specific button is held down. When right mouse is down the camera class will take the mouse information and store the old position, using both variables to determine how much to rotate the camera by.

In the early stages of working on the camera an issue arose where even if the mouse wasn't moving the camera would slowly rotate, the cause of this issue was how the mouse position was being updated. The mouse deltas would be calculated but when the mouse stopped moving the camera would still rotate between the last recorded movement without checking whether the mouse position had actually changed before applying a new rotation value.

Mouse Picking

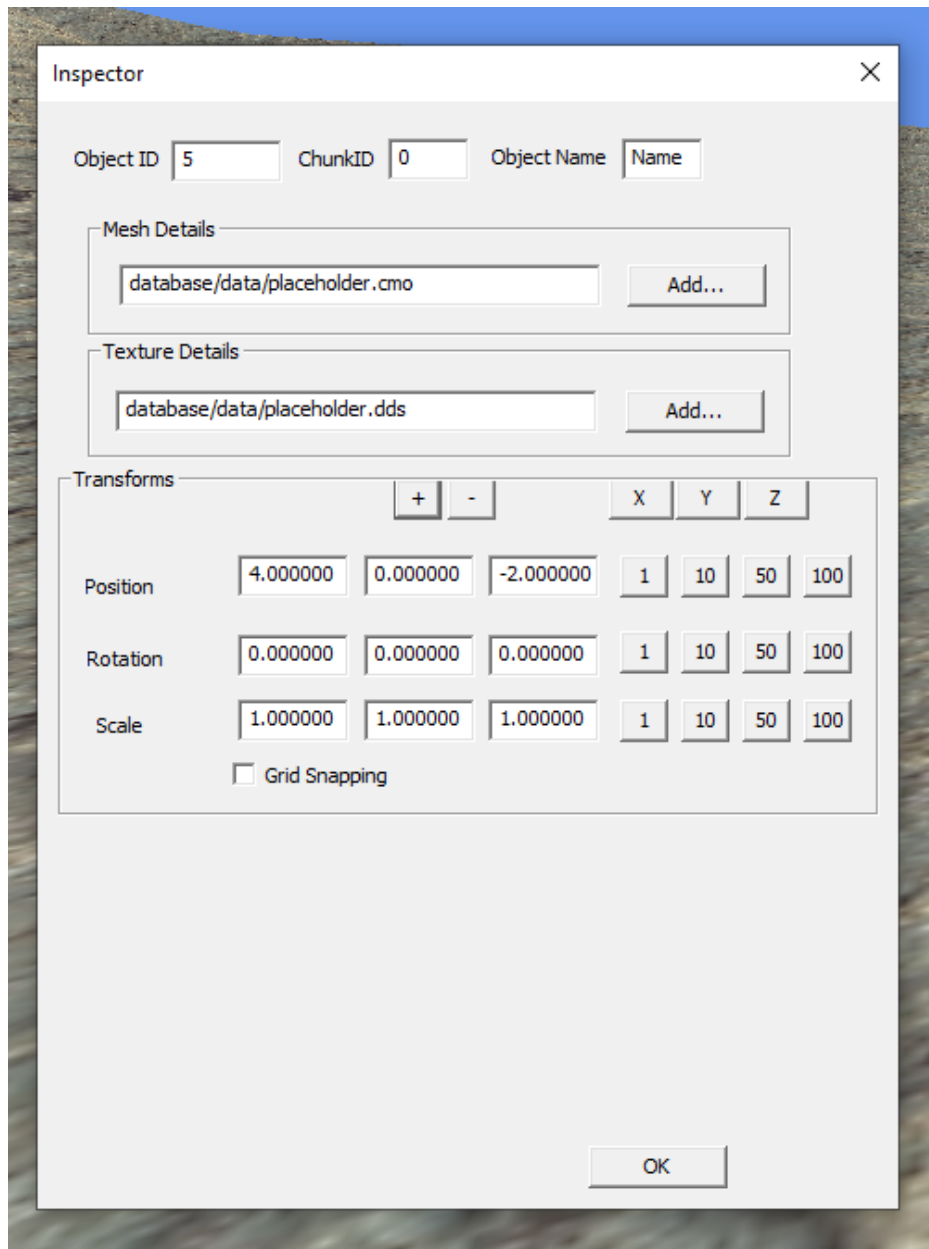
Basic mouse picking is a requirement of the coursework. This has been further developed by myself to allow for multiselect of objects in the scene. The mouse picking makes the dialogue box for selection obsolete as it is easier for the user to click on objects in the window as opposed to navigating the window and scrolling down to find an object based solely on an ID they might not know. The multiselect works by casting a ray from the mouse screen coordinates to the scene world position between the near and far planes. Any objects in this area will be found. If the position intersects the collision box of an object that object is set as the currently selected object.

Before a new ID is pushed into the array of ints we first check if the user is holding down shift, this enables multiselect. If multiselect is enabled the ID is pushed back into the vector or if it is already selected it is removed from the array. If multiselect is disabled, the vector is cleared and the new ID is pushed back.

While the current implementation works, a possible expansion for more accurate picking would be to determine object distances from the mouse point, as currently at certain angles the selected object will often times be the last object in a line.

Inspector Window

The MFC inspector window shows key details about a selected object, including ID, Chunk ID, Name and transforms. This dialogue box can't be opened unless there is at least one selected object at that moment. Each field in the window is populated by that object using the UpdateFromSelectedObject function. This function finds the current object from the scenegraph and gets all values required and places them into their correct place in the window. There are buttons at the top to determine what axis you are adjusting and whether it is by a positive or negative amount, beside each box there is a number (1,10,50,100), this number is used to apply that amount to the object.



This makes use of a switch statement that checks the button ID and applies a value accordingly, alongside storing what the current transform is that is being applied into an enum.

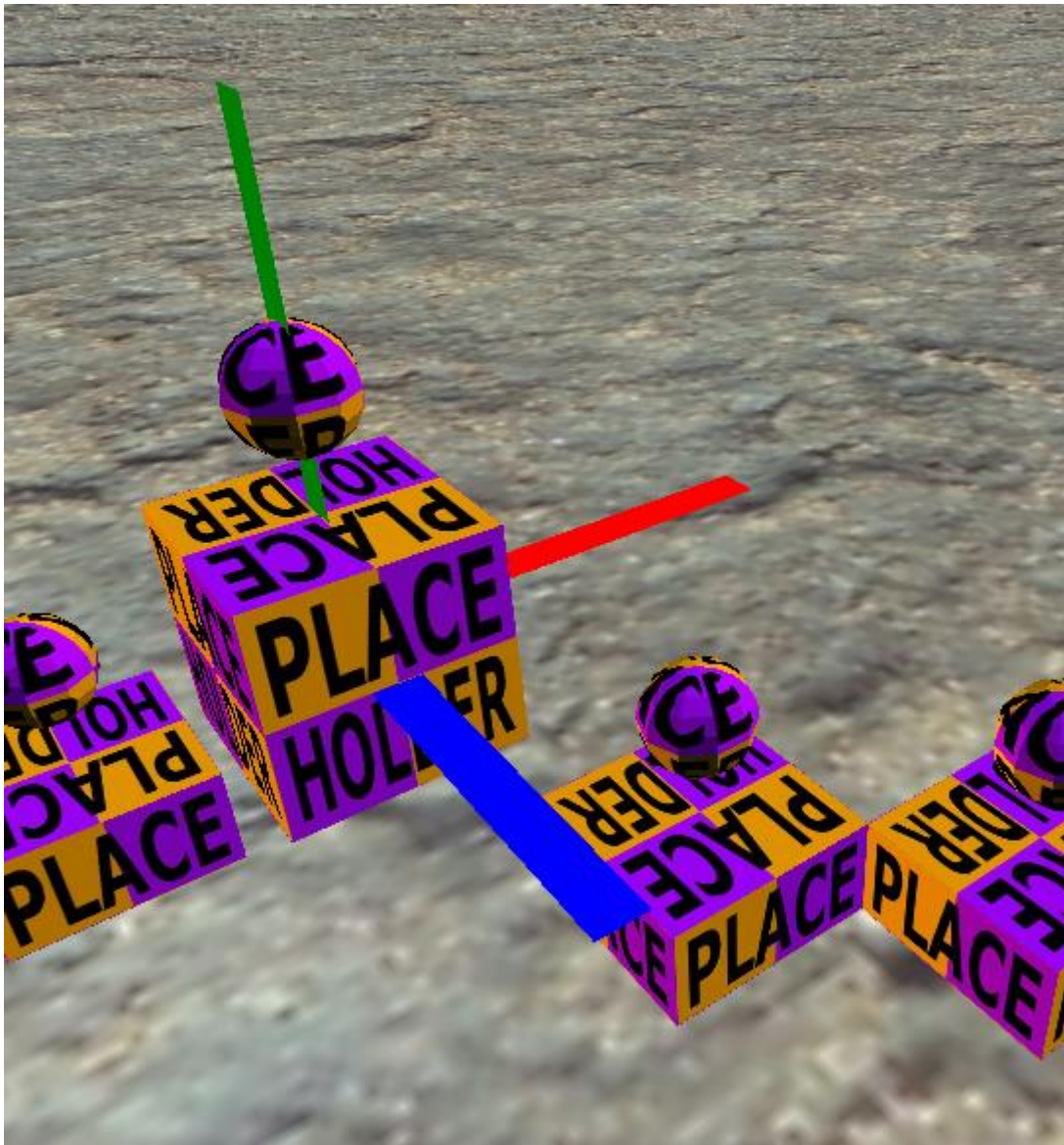
Engine Settings

The next MFC dialogue box is used as a savable and loadable engine settings area. This currently includes variables for controlling the camera such as speed, rotation and inversion of controls but this could be expanded to include many more things like screen size or resolution. When any variables are changed inside this window, they are passed

through toolmain, to game, to the camera class to update those variables. After the values are saved to a .txt file to be loaded upon restarting of the engine. Any changes made in the editor settings will be saved and loaded any time the game begins.

Gizmo

The gizmo is rendered and placed at the centre of the most recently selected object. It supports three modes of transformation, location, rotation and scale. These modes are toggled by pressing the 1-3 keys on the keyboard which modifies the m_mode state. Each mode has different logic to determine how the gizmo effects the object.



When a user clicks near any of the gizmo axis lines a picking function is used to determine which axis has been clicked on using ray segment intersection logic to determine the closest

axis to the mouse position. When an axis is selected the dragging bool is set to true which tracks mouse movement and applies it to the axis and mode currently selected by calculating the delta between the old and new mouse position.

The gizmo is rendered using a primitive batch and vertexcolour position with each axis being a certain colour:

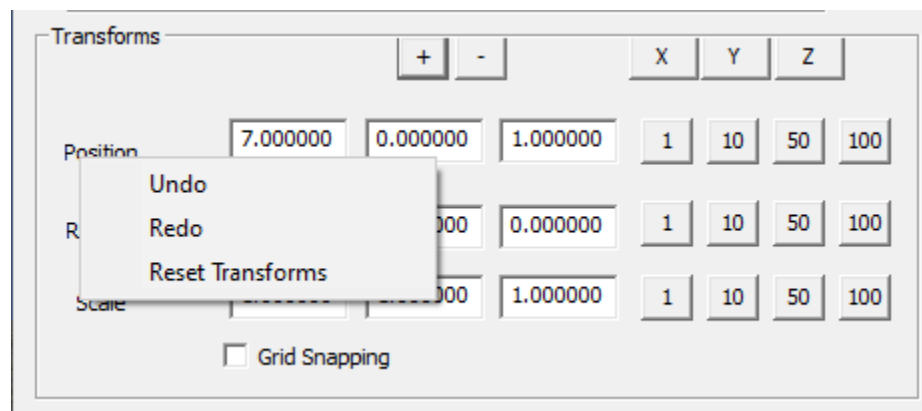
- X = red
- Y = blue
- Z = green

Undo / Redos

The player has the option to undo or redo any changes they have made to object transforms, this is a pivotal aspect of any engine as mistakes can happen, and this is the fastest way to revert those mistakes. The undo/redo works through the use of command interfaces. A base class with abstract functions for Execute, Undo and Redo is used and extended for different functionalities.

A struct is used to store transform data, this is written to and pushed onto a command manager stack. This stack allows for the user to press the undo command and revert to the previous state, saving the recently undone command in a redo stack in case the user wishes to redo that change. The command manager works by taking in a child class of the base command interface, before calling its execute function to fulfill whatever command it is assigned to do before storing that command information onto the undo stack.

This can be utilised both in the editor and in the MFC window, in MFC a context menu has been created to allow for undo and redo while using the hotkey control + z or y will undo or redo the changes made to an objects transforms.



Cut/Copy/Paste/Delete

The command interfaces have been expanded to include additional functionality, this include the ability to cut, copy, paste and delete objects from the scene – while being able to undo and redo each of these functions. These would allow designers to easily make copies of work or delete different objects in the world without having to edit the code, abstracting the complex implementations from them and making their job easier. I was inspired to do this as it is common in everything from maya to unreal to Microsoft word.

As stated, these are implementations of command patterns, each action is encapsulated in its own pattern and stored in the manager. This means by default they all support undo and redo.

Cut Command:

Execute stored a deep copy of the selected object and remove it from the scene graph

Undo will add that copy back into the scene graph at the original index if possible

Redo would remove it again

This pattern stores a pointer to the object for use in the paste command

Copy Command:

This doesn't incorporate a command, instead it stores a deep copy of the selected object in a toolmain variable `m_copiedObject`.

This is similar to tools like photoshop as copying just prepares the data for the paste operation

Paste Command:

Execute clones the stored object and push it back into the scene graph with a new ID.

Undo pops the object from the scene graph, removing it from the scene.

Redo reinserts the object using a stored clone in the manager and paste pattern.

This supports both copy and cut patterns by checking for a boolean `m_hascopieddata` or the last cut command to determine which is needed.

Delete Command:

Execute stores and removes the selected object

Undo adds the object to the scene at its original index if possible or at the end if that index is invalid.

Redo reinserts into place using the saved clone

Similar to undo and redo, each of the patterns interacts with the command manager which uses two stacks to track the history of the commands. After every alteration to an object, a new command is pushed back which allows for undo or redo to be called.

Critical Analysis

What went Right?

I found that learning MFC was a very fun experience. I understand that MFC is considered outdated, but many companies still use outdated systems due to the effort and complexity of upgrading or switching everything over. I enjoyed creating the dialogue boxes and it was quite easy to understand how to communicate data between the game, toolmain, mfcmain and the dialogue box. On the other hand I found this to be quite annoying at times, to change camera values in the editor settings you needed to call toolmain to call the game class which gets the camera from the camera class to get and set the values.

I was very proud of my command patterns as I had never known about them before researching them for this module submission. I see them as a valuable piece of knowledge to take forward into my future work because they are so useful in many occasions and are a great way of making code cleaner as each operation is in its own separate place and is interacted with through a line of code anywhere with the command manager.

What went Wrong?

Communication between Toolmain, MFCMain and Game was frustrating at times, and with more time available to myself I would have tried to find a better way of implementing the features. I personally also had logic issues when trying to figure out object selection and highlighting for the gizmo, making use of an object ID in some places but the index array in the scene graph for others which led to a lot of wasted time debugging these issues.

I also spent a lot of time debugging mouse picking thinking it was buggy due to me resizing the screen. What was required was integrating the Toolmain and Game classes with the MFC OnWindowPosChanged and OnSize to pass in the new screen bounds and centre each time a player would resize the window, after this everything worked perfectly. This took a lot of time as I was unfamiliar with MFC and its standard functions.

How I would do things differently

I would attempt to refactor the architecture to reduce chained function calls, perhaps implementing something similar to the Unreal Engine delegate system or C++ event system. I would fully implement the inspector to allow for more live changes of object values such as the pivot point or the different booleans in SceneObject.

I would setup arc camera functionality, I had a half working version in before submission I felt it wasn't a high enough quality to remain in and as such removed it. Being able to jump to a selected object to focus on it would be beneficial if the editor environment grew in size as designers could be going from one area to another making changes.

I would implement grid snapping for object in the level as this is a common feature in any engine due to the fine nature of placement at times.

Finally I would improve the engine settings dialogue box save/load functionality. Currently it writes to a .txt file with the fstream library which is functional but not as optimised as it could be in terms of performance or structure. For more efficient and faster read/writing speeds I would look into using a different format, some ideas include:

- JSON files typically for game states, user data and save data.
- INI files which is typical for game engine or game settings.

Currently the system isn't large enough to have that big of an impact on performance but as the system grows an alternative method would be required to handle all the settings that would be used in an editor.

References:

<https://github.com/microsoft/DirectXTK/wiki/Getting-Started>

https://www.tutorialspoint.com/mfc/mfc_overview.htm

<https://learn.microsoft.com/en-us/cpp/mfc/mfc-desktop-applications?view=msvc-170>

Huge thanks to Denis for explaining command patterns to me.

<https://www.youtube.com/watch?v=yDkIK3JfHkw>